

RESEARCH ARTICLE

AutoML-Pipeline: A RAG-Enhanced Code Generation Framework With Pre-Validation for Cloud-Native Machine Learning Workflows

WENYU ZHAO¹, TINGJIE CHEN², JIE SI YANG³,
AND LEI QIU⁴, (Member, IEEE)

¹Microsoft, Beijing 100080, China

²Intel, Shanghai 200131, China

³The University of Utah, Salt Lake City, UT 84112, USA

⁴Ningbo University of Technology, Ningbo 315104, China

Corresponding author: Wenyu Zhao (wenyuzhao@stu.xidian.edu.cn)

ABSTRACT The proliferation of cloud-native machine learning platforms has significantly accelerated model development and deployment cycles. However, constructing and maintaining heterogeneous pipeline code spanning multiple languages (Python, YAML, Spark SQL) and cloud-specific configurations remains labor-intensive and error-prone. Existing LLM-based code generation tools lack awareness of runtime constraints and historical execution patterns, frequently producing code with resource misconfigurations or dependency conflicts that fail upon deployment. To address these challenges, we propose *AutoML-Pipeline*, a closed-loop code generation framework that integrates Retrieval-Augmented Generation (RAG) with reinforcement learning feedback mechanisms. Our approach leverages a knowledge base constructed from successful pipeline execution logs to guide GPT-4 in generating deployment-ready code that adheres to platform-specific constraints. The key innovation lies in a novel *Pre-validation Agent* that employs simulated execution environments to predict resource consumption and detect dependency conflicts before actual deployment. This agent iteratively refines generated code through a feedback loop informed by predicted execution profiles and dependency graphs. We evaluate our framework on the CodeSearchNet dataset augmented with Azure ML pipeline specifications, demonstrating a 43.7% improvement in first-submission success rate and 31.2% reduction in resource over-provisioning compared to vanilla GPT-4 baselines. Ablation studies confirm that both the RAG retrieval mechanism and pre-validation agent contribute substantially to performance gains. Our work establishes a practical paradigm for integrating large language models with domain-specific runtime intelligence, with potential applications extending to other infrastructure-as-code generation tasks.

INDEX TERMS Code generation, retrieval-augmented generation, cloud computing, automated optimization, large language models.

I. INTRODUCTION

The advent of cloud-native machine learning platforms has fundamentally transformed the landscape of model development and deployment [1], [2]. Platforms such as Azure

Machine Learning (Azure ML) [3], Amazon SageMaker [4], and Google Vertex AI [5] enable data scientists to orchestrate complex workflows spanning data preprocessing, distributed training, hyperparameter optimization, and model serving. However, this convenience comes at a cost: practitioners must navigate heterogeneous programming paradigms, including Python scripts for model logic, YAML configurations for

The associate editor coordinating the review of this manuscript and approving it for publication was Wenbing Zhao¹.

pipeline definitions, and domain-specific languages like Spark SQL for data transformations [6], [7].

Recent studies reveal that data scientists spend approximately 60-80% of their time on pipeline engineering and deployment tasks rather than core algorithmic innovation [8], [9]. Manual construction of these pipelines is not only time-consuming but also highly error-prone. Common failure modes include resource misconfiguration (e.g., insufficient memory allocation for GPU-intensive tasks), dependency conflicts across heterogeneous runtime environments, and subtle incompatibilities between framework versions [10], [11]. A survey of production ML systems found that over 40% of pipeline failures stem from configuration errors rather than algorithmic issues [12].

The emergence of large language models (LLMs) such as GPT-4 [13], CodeLlama [14], and StarCoder [15] has opened new possibilities for automated code generation. These models demonstrate remarkable proficiency in generating syntactically correct code across multiple programming languages [16], [17]. However, their application to cloud-native ML pipeline generation remains challenging. First, vanilla LLMs lack awareness of platform-specific constraints and runtime semantics [18]. For instance, Azure ML pipelines require explicit declarations of compute targets, environment dependencies, and data references that must align with the cloud infrastructure state. Second, LLMs trained on general-purpose code repositories often generate solutions that are syntactically valid but operationally infeasible due to resource constraints or deprecated API usage [19], [20].

Retrieval-Augmented Generation (RAG) has emerged as a promising paradigm to ground LLM outputs in domain-specific knowledge [21], [22]. By retrieving relevant examples from curated knowledge bases, RAG systems can guide generation toward solutions that have proven successful in similar contexts [23], [24]. However, existing RAG approaches for code generation primarily focus on semantic similarity matching without considering runtime execution profiles [25], [26]. This limitation is particularly acute in ML pipeline generation, where success depends not only on functional correctness but also on efficient resource utilization and compatibility with dynamic cloud environments.

To address these challenges, we propose *AutoML-Pipeline*, a closed-loop framework that synergistically combines RAG with reinforcement learning feedback to generate optimized cloud-native ML pipeline code. Our framework constructs a knowledge base from historical pipeline execution logs, capturing not only code artifacts but also associated resource consumption patterns, execution traces, and failure diagnostics. During generation, we retrieve relevant pipeline templates and execution profiles to contextualize the LLM's output. The key innovation of our work is a *Pre-validation Agent* that employs lightweight simulation to predict resource requirements and detect dependency conflicts before code

submission. This agent operates in an iterative refinement loop, providing corrective feedback to the LLM until convergence criteria are met.

Our contributions can be summarized as follows:

- We propose a novel RAG-enhanced code generation framework specifically designed for cloud-native ML pipelines, integrating historical execution intelligence into the generation process. To the best of our knowledge, this is the first work to systematically incorporate runtime profiling data into LLM-based pipeline code generation.
- We design a Pre-validation Agent that combines static dependency analysis with lightweight execution simulation to predict resource consumption and identify configuration errors before deployment. This agent employs a graph-based dependency resolver and a learned cost model to iteratively refine generated code.
- We introduce a reinforcement learning feedback mechanism that updates retrieval policies based on deployment outcomes, progressively improving the relevance of retrieved examples over time. This creates a self-improving system that adapts to evolving platform characteristics and user requirements.
- We conduct comprehensive experiments on the CodeSearchNet dataset augmented with Azure ML pipeline specifications, demonstrating substantial improvements over state-of-the-art baselines. Our framework achieves a 43.7% increase in first-submission success rate, 31.2% reduction in resource over-provisioning, and 27.8% faster convergence in iterative refinement compared to vanilla GPT-4 and existing code generation tools.
- We perform extensive ablation studies to validate the contribution of each component and provide qualitative analysis of failure cases, offering insights into limitations and future research directions.

The remainder of this paper is organized as follows. Section II reviews related work in code generation, retrieval-augmented LLMs, and ML pipeline automation. Section III provides preliminary background on cloud-native ML platforms and the technical challenges they present. Section IV details our methodology, including the RAG architecture, pre-validation agent design, and reinforcement learning feedback loop. Section V presents experimental setup, results, and analysis. Finally, Section VI concludes with a discussion of limitations and future work directions, including potential applications to other infrastructure-as-code generation domains such as Kubernetes configurations and CI/CD pipeline definitions.

II. RELATED WORKS

Our work intersects several research domains, including automated code generation with large language models, retrieval-augmented generation systems, and machine learning pipeline automation. We organize related work into three main categories.

A. CODE GENERATION WITH LARGE LANGUAGE MODELS

The application of large language models to code generation has witnessed rapid progress in recent years. Early neural approaches such as seq2seq models [27] and tree-based decoders [28] demonstrated promising results on narrow benchmarks but struggled with complex, real-world programming tasks. The introduction of pre-trained models like CodeBERT [29] and GraphCodeBERT [30] marked a significant advancement by leveraging large-scale code corpora for representation learning. More recent large-scale models have achieved remarkable performance on diverse code generation tasks. Codex [16], built upon GPT-3 architecture, demonstrated strong few-shot learning capabilities and powered GitHub Copilot. AlphaCode [31] achieved competitive results in programming competitions through large-scale sampling and filtering. Subsequent models such as CodeGen [32], InCoder [33], and CodeT5+ [34] have further pushed the boundaries of automated programming assistance. Despite these advances, existing approaches face limitations when applied to domain-specific code generation tasks. Jain et al. [35] noted that general-purpose code LLMs often generate syntactically correct but semantically incorrect code in specialized domains. Cassano et al. [36] demonstrated that incorporating domain-specific knowledge through fine-tuning improves performance but requires substantial labeled data. Our work addresses this challenge by augmenting pre-trained LLMs with retrieval from domain-specific execution logs, avoiding expensive fine-tuning while achieving specialization. Several recent works have explored LLM applications in software engineering workflows beyond basic code completion. Nashid et al. [37] proposed retrieval-augmented prompting for API usage generation, while Shrivastava et al. [38] investigated repository-level code generation. However, these approaches focus primarily on functional correctness and do not address operational concerns such as resource allocation and runtime compatibility, which are critical for cloud-native deployments.

B. RETRIEVAL-AUGMENTED GENERATION AND KNOWLEDGE-INTENSIVE TASKS

Retrieval-Augmented Generation has emerged as an effective paradigm for grounding language model outputs in external knowledge sources. The seminal RAG framework [21] demonstrated that retrieving relevant passages from Wikipedia significantly improves performance on knowledge-intensive NLP tasks. Subsequent work has explored diverse retrieval strategies, including dense retrieval with learned encoders [39], iterative retrieval with self-refinement [40], and multi-hop reasoning over retrieved documents [41]. In the context of code generation, several systems have incorporated retrieval mechanisms to improve output quality. CodeRetriever [25] retrieves relevant code snippets from documentation to guide neural code summarization and generation. DocPrompting [26]

dynamically retrieves API documentation to provide context for LLM-based code generation. ReACC [42] combines retrieval with adaptive code completion to improve suggestion relevance. These approaches primarily focus on semantic similarity-based retrieval and do not explicitly model execution-level constraints. More recent work has begun to explore execution-aware code generation. CodeRL [43] employs reinforcement learning with execution feedback to improve code correctness. CodeT [44] uses test cases to guide iterative code refinement. However, these methods require explicit test cases or oracles, which are often unavailable in infrastructure code generation scenarios. Our pre-validation agent addresses this gap by simulating execution environments to predict runtime behavior without requiring ground-truth test cases. Knowledge base construction for code generation has also received attention. CoSQA [45] curated a large-scale question-answering dataset for code search. CodeXGLUE [46] provides a comprehensive benchmark suite for code intelligence tasks. However, existing knowledge bases primarily contain code snippets and natural language descriptions, lacking execution profiles and resource consumption data that are essential for cloud deployment scenarios. Our work constructs a knowledge base that explicitly captures runtime characteristics alongside code artifacts.

C. MACHINE LEARNING PIPELINE AUTOMATION AND MLOps

The automation of machine learning pipelines has been a central focus of the MLOps community. Early systems like TFX [47] and Kubeflow [48] provided workflow orchestration frameworks but required significant manual configuration. MLflow [2] introduced experiment tracking and model registry capabilities, while platforms like Metaflow [49] and ZenML [50] offered higher-level abstractions for pipeline definition. AutoML systems have automated various aspects of the ML workflow, including neural architecture search [51], hyperparameter optimization [52], and feature engineering [53]. However, these systems typically focus on model selection and training rather than end-to-end pipeline code generation. Recent work on ML pipeline synthesis includes Plumber [54], which automatically constructs data preparation pipelines, and AutoPandas [55], which synthesizes data transformation code. These approaches operate at a higher level of abstraction and do not generate deployable cloud-native pipeline code. Several commercial platforms have introduced AI-assisted pipeline development features. Azure ML Designer [3] provides a visual interface for pipeline construction, while Amazon SageMaker Autopilot [56] automates model training workflows. However, these tools primarily rely on template-based generation and manual configuration, lacking the flexibility of LLM-based code generation. Recent research has explored failure prediction and debugging in ML pipelines. Karlaš et al. [57] proposed nearest neighbor-based debugging

for data validation failures. Grafberger et al. [58] developed automated inspection tools for ML pipelines. Schelter et al. [59] investigated data quality testing automation. While these works address important operational concerns, they focus on post-deployment diagnosis rather than proactive error prevention during code generation. Our work differs from existing approaches in several key aspects. First, we target the end-to-end generation of heterogeneous pipeline code spanning multiple languages and cloud-specific configurations. Second, we integrate execution intelligence from historical deployments into the generation process through RAG. Third, we introduce a pre-validation mechanism that predicts and prevents common deployment failures before code submission. To our knowledge, this is the first work to systematically combine LLM-based code generation with runtime-aware validation for cloud-native ML pipelines.

III. PRELIMINARIES

In this section, we provide necessary background on cloud-native ML pipelines and formally define the code generation problem. We introduce key concepts and notations that will be used throughout the paper.

A. CLOUD-NATIVE ML PIPELINE STRUCTURE

A cloud-native machine learning pipeline $\mathcal{P}$ consists of a directed acyclic graph (DAG) $G = (V, E)$, where each vertex $v \in V$ represents a computational step (e.g., data preprocessing, model training, evaluation) and each edge $e \in E$ denotes a data dependency between steps. Formally, we define:

$$\mathcal{P} = \langle G, \mathcal{C}, \mathcal{D}, \mathcal{R} \rangle, \quad (1)$$

where: $G = (V, E)$ is the execution graph with n steps; $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$ is the set of code artifacts, where c_i implements step v_i and may be written in different languages (Python, YAML, Spark SQL); $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$ represents data dependencies including input datasets, intermediate outputs, and environment configurations; $\mathcal{R} = \{r_1, r_2, \dots, r_k\}$ specifies resource requirements, where $r_i = \langle \text{compute}, \text{memory}, \text{storage} \rangle$. Each code artifact c_i must conform to platform-specific APIs and constraints. For example, in Azure ML, a pipeline step requires declarations of compute targets, runtime environments (including software dependencies), and data references that must exist in the platform registry.

B. PROBLEM FORMULATION

Given a natural language specification $\mathcal{S}$ describing the desired ML workflow and a cloud platform context $\mathcal{X}$ (including available compute resources, registered environments, and accessible data assets), the objective is to generate a complete executable pipeline $\mathcal{P}^*$ that satisfies three key

requirements:

$$\mathcal{P}^* = \arg \max_{\mathcal{P}} [\alpha \cdot \mathbb{I}_{\text{correct}}(\mathcal{P}) + \beta \cdot \mathbb{I}_{\text{viable}}(\mathcal{P}) + \gamma \cdot \text{Eff}(\mathcal{P})], \quad (2)$$

where: $\mathbb{I}_{\text{correct}}(\mathcal{P}) \in \{0, 1\}$ indicates whether $\mathcal{P}$ correctly implements the workflow specification $\mathcal{S}$; $\mathbb{I}_{\text{viable}}(\mathcal{P}) \in \{0, 1\}$ indicates whether $\mathcal{P}$ can be successfully deployed on the target platform without configuration errors; $\text{Eff}(\mathcal{P}) \in [0, 1]$ measures resource efficiency, penalizing over-provisioning and rewarding appropriate resource allocation; α, β, γ are weight parameters balancing these objectives. In practice, deployment viability $\mathbb{I}_{\text{viable}}$ is critical: a pipeline that fails to deploy provides no value regardless of its functional correctness. Common deployment failures include: (1) referencing non-existent compute targets or environments, (2) dependency version conflicts, (3) insufficient resource allocation causing out-of-memory errors, and (4) incorrect data path specifications.

C. KEY CHALLENGES

Automated generation of cloud-native ML pipelines faces several inherent challenges:

Heterogeneous Code Synthesis. A single pipeline requires coordinating multiple programming paradigms. Training logic is typically written in Python using ML frameworks, data transformations may use Spark SQL, and orchestration is defined in YAML. These components must maintain semantic consistency in data schemas and type compatibility across language boundaries.

Platform Constraint Satisfaction. Cloud platforms impose complex constraints that are often implicit or under-documented. For instance, certain compute SKUs may only support specific regions, environment configurations must declare all dependencies with compatible versions, and data references must use platform-specific URI schemes. Violating any constraint results in deployment failure.

Resource Specification. Determining appropriate resource allocations requires understanding workload characteristics. A vision model training job may require GPUs with sufficient VRAM, while a data preprocessing step may be memory-bound but CPU-intensive. Misconfiguration leads to either job failures or cost inefficiency.

Lack of Execution Feedback. Traditional code generation models operate in an open-loop fashion, producing code without validation against runtime constraints. This results in high failure rates when deployed, as the models cannot anticipate platform-specific execution behaviors.

These challenges motivate the need for generation frameworks that integrate domain-specific knowledge and runtime awareness, which we address through our proposed methodology in the next section.

IV. METHODOLOGY

We present AutoML-Pipeline, a closed-loop framework for automated generation of cloud-native ML pipeline

code. Our approach addresses the key challenges identified in Section III through three synergistic components: a retrieval-augmented generation engine that leverages historical execution intelligence, a pre-validation agent that predicts and prevents deployment failures, and a reinforcement learning mechanism that continuously improves retrieval quality.

A. FRAMEWORK OVERVIEW

The core insight driving our framework is that successful pipeline deployments exhibit recurring patterns in their code structure, resource configurations, and dependency specifications. By systematically extracting and indexing these patterns from historical execution logs, we can guide an LLM to generate code that not only satisfies functional requirements but also adheres to platform-specific operational constraints.

Our framework operates in a closed loop consisting of three phases. First, given a natural language task specification, we retrieve relevant historical pipeline examples along with their execution profiles from a curated knowledge base. Second, these retrieved examples augment the LLM prompt to generate an initial pipeline draft. Third, a pre-validation agent simulates the execution environment to predict potential failures and provides corrective feedback. This feedback is incorporated into subsequent generation iterations until the pipeline passes all validation checks or reaches a predefined iteration limit. Throughout this process, a reinforcement learning mechanism observes actual deployment outcomes and adjusts the retrieval policy to prioritize examples that lead to successful generations.

B. KNOWLEDGE BASE CONSTRUCTION AND INDEXING

The foundation of our approach is a knowledge base $\mathcal{K}$ constructed from historical pipeline execution logs. Each entry $k_i \in \mathcal{K}$ encapsulates not only the pipeline code but also rich metadata about its execution behavior. Specifically, each entry is a structured tuple:

$$k_i = \langle \mathcal{P}_i, \mathcal{E}_i, \mathcal{M}_i, \mathcal{O}_i \rangle. \quad (3)$$

The pipeline artifact $\mathcal{P}_i$ contains all code components across different languages, preserving the complete implementation. The execution profile $\mathcal{E}_i$ records resource consumption metrics including CPU utilization curves, memory footprints, GPU usage patterns, and I/O throughput statistics collected during actual execution. The metadata $\mathcal{M}_i$ includes the natural language description of the pipeline's purpose, platform configurations such as compute cluster specifications and software environment definitions, and the dependency graph capturing library versions and their compatibility constraints. The outcome label $\mathcal{O}_i$ indicates whether the deployment succeeded, and for failed cases, includes diagnostic error messages and stack traces.

We deliberately focus the knowledge base on successfully deployed pipelines, as these represent validated solutions that satisfy both functional and operational requirements.

Failed executions are retained selectively when their error diagnostics provide instructive negative examples for common pitfalls. To enable efficient retrieval, we encode each entry using a dual-embedding strategy. The task description component of $\mathcal{M}_i$ is embedded using a pre-trained sentence transformer to capture semantic similarity. Simultaneously, we construct a graph-based embedding for the dependency structure, representing libraries and their version constraints as nodes and compatibility relationships as edges. This graph is embedded using a graph neural network to capture structural patterns in dependency configurations.

The retrieval function operates by computing a weighted similarity score that combines semantic and structural matching:

$$\text{score}(q, k_i) = \lambda \cdot \text{sim}_{\text{semantic}}(q, \mathcal{M}_i) + (1 - \lambda) \cdot \text{sim}_{\text{structural}}(\mathcal{X}_q, \mathcal{E}_i), \quad (4)$$

where q represents the query consisting of the task specification and platform context $\mathcal{X}_q$, and λ balances the contribution of semantic versus structural similarity. The semantic similarity measures how closely the query's intent aligns with historical tasks, while structural similarity assesses whether the query's platform context is compatible with the retrieved example's execution environment. This dual-similarity approach ensures that retrieved examples are not only functionally relevant but also operationally compatible with the target deployment environment.

C. RETRIEVAL-AUGMENTED CODE GENERATION

Given a task specification $\mathcal{S}$ and platform context $\mathcal{X}$, the generation process begins by retrieving the top- k most relevant entries from the knowledge base. These entries are denoted as $\mathcal{K}_{\text{ret}} = \{k_{i_1}, k_{i_2}, \dots, k_{i_k}\}$ where indices are ordered by descending similarity scores. The retrieved examples serve dual purposes: they provide concrete code templates demonstrating platform-specific API usage patterns, and their execution profiles inform resource allocation decisions.

We construct an augmented prompt $\mathcal{T}_{\text{prompt}}$ that interleaves the task specification with retrieved examples in a carefully designed format. The prompt structure follows a few-shot learning paradigm, where each retrieved example is presented as a triplet of task description, generated code, and key execution statistics. Importantly, we extract and highlight critical execution characteristics such as peak memory usage and average CPU load from the execution profiles, enabling the LLM to make informed decisions about resource specifications. The prompt also includes explicit instructions emphasizing platform constraints, such as the requirement to reference only pre-registered compute targets and to declare all dependencies with specific version pins.

The base LLM, which we instantiate using GPT-4, processes this augmented prompt to generate an initial pipeline draft $\mathcal{P}_0$. The generation is configured to produce complete, multi-file outputs including Python training scripts, YAML pipeline definitions, and any auxiliary configuration files.

We employ constrained decoding to enforce syntactic validity within each language component, though this does not guarantee semantic correctness or operational viability.

D. PRE-VALIDATION AGENT DESIGN

The pre-validation agent represents the key innovation of our framework, bridging the gap between syntactic code generation and operational deployment. Traditional LLM-based code generators operate in an open-loop fashion, producing code without verification against runtime constraints. Our agent introduces a closed-loop validation mechanism that predicts deployment failures before actual submission, enabling iterative refinement.

The agent employs two complementary analysis techniques. First, a static dependency analyzer constructs a dependency graph $G_{\text{dep}} = (N, A)$ from the generated code, where nodes N represent software packages and edges A represent dependency relationships with version constraints. This graph is checked for conflicts by formulating the version satisfiability problem as a constraint satisfaction problem (CSP). Specifically, for each package $p \in N$, we collect all version constraints imposed by dependent packages. A conflict is detected if:

$$\bigcap_{c \in \text{constraints}(p)} \text{versions}(p, c) = \emptyset, \quad (5)$$

where $\text{versions}(p, c)$ denotes the set of versions of package p satisfying constraint c . When conflicts are detected, the agent identifies the minimal set of conflicting constraints and suggests compatible version resolutions.

Second, a resource estimation model predicts the computational requirements of the generated pipeline. This model is learned from the execution profiles in the knowledge base. For a given pipeline step v_i with code c_i , we extract code features including the presence of specific framework calls (e.g., `torch.nn.DataParallel` indicating multi-GPU training), loop structures over dataset iterations, and tensor dimension specifications. These features are fed into a gradient boosting regressor that predicts resource consumption:

$$\hat{r}_i = f_{\text{predict}}(\phi(c_i), \mathcal{E}_{\text{similar}}), \quad (6)$$

where $\phi(c_i)$ extracts code features and $\mathcal{E}_{\text{similar}}$ aggregates execution profiles from similar historical pipelines. The predicted resource vector $\hat{r}_i = \langle \hat{\text{cpu}}_i, \hat{\text{mem}}_i, \hat{\text{gpu}}_i \rangle$ is compared against the resource specification in the generated code. If the predicted requirements exceed the allocated resources by a margin ϵ , the agent flags a potential under-provisioning issue.

The agent synthesizes findings from both analysis techniques into structured feedback. For dependency conflicts, it provides specific version recommendations that resolve all constraints. For resource misconfigurations, it suggests adjusted compute target specifications or memory allocations with quantitative justifications based on predicted consumption. This feedback is formatted as natural language

corrections that are appended to the original task specification, forming an enhanced prompt for the next generation iteration:

$$\mathcal{T}_{\text{prompt}}^{(t+1)} = \mathcal{T}_{\text{prompt}}^{(t)} \oplus \text{Feedback}(\mathcal{P}_t), \quad (7)$$

where $\oplus$ denotes prompt concatenation and $\text{Feedback}(\mathcal{P}_t)$ is the agent's analysis of the t -th iteration's output. This iterative refinement continues until the agent's validation checks all pass, indicating high confidence in deployment viability, or a maximum iteration count is reached.

E. REINFORCEMENT LEARNING FEEDBACK LOOP

While the pre-validation agent improves deployment viability through simulation, it cannot perfectly predict all runtime behaviors. Actual deployment outcomes provide invaluable supervision signals that we leverage to continually improve the retrieval policy. We formulate this as a contextual bandit problem where the action is the selection of retrieval parameters and the reward is derived from deployment success.

Specifically, the retrieval policy is parameterized by the similarity weighting factor λ in Equation (2) and the retrieval count k . Different task categories may benefit from different balances between semantic and structural matching. For instance, data preprocessing pipelines might prioritize structural similarity to ensure compatible data schemas, while model training pipelines might prioritize semantic similarity to match algorithmic approaches. Rather than using fixed parameters, we learn a policy π_θ that maps task characteristics to optimal retrieval configurations:

$$(\lambda^*, k^*) = \pi_\theta(\mathcal{S}, \mathcal{X}). \quad (8)$$

The policy network π_θ is implemented as a multi-layer perceptron that takes as input embeddings of the task specification and platform context. It outputs a continuous value for $\lambda \in [0, 1]$ and a discrete value for $k \in \{1, 2, \dots, K_{\text{max}}\}$. We train this policy using the REINFORCE algorithm with deployment outcome as the reward signal. Successful deployments that execute without errors yield a reward of $+1$, while failed deployments yield a reward of -1 . Additionally, we incorporate a shaped reward component that considers resource efficiency:

$$R(\mathcal{P}) = \mathbb{I}_{\text{success}}(\mathcal{P}) + \eta \cdot \left(1 - \frac{\text{allocated} - \text{actual}}{\text{allocated}}\right), \quad (9)$$

where the second term penalizes over-provisioning and η controls the weight of efficiency considerations. The policy gradient update follows:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [R(\mathcal{P}) \cdot \nabla_\theta \log \pi_\theta(\lambda, k | \mathcal{S}, \mathcal{X})]. \quad (10)$$

This learning process occurs continuously as the system processes new generation requests and observes their deployment outcomes. Over time, the policy adapts to shifts in platform characteristics, such as the introduction of new compute SKUs or framework updates, ensuring the retrieval strategy remains effective as the environment evolves.

F. END-TO-END GENERATION ALGORITHM

Algorithm 1 summarizes the complete generation process. The algorithm accepts a task specification and platform context as input and outputs a validated pipeline ready for deployment. The retrieval policy determines the optimal parameters for querying the knowledge base, and the iterative refinement loop continues until validation succeeds or the iteration budget is exhausted. Upon successful generation, the deployment outcome is logged to update both the knowledge base and the retrieval policy, creating a self-improving system.

The modular design of our framework allows each component to be independently improved. The knowledge base can be enriched with additional execution logs, the pre-validation agent can incorporate more sophisticated analysis techniques, and the retrieval policy can be adapted to specific organizational deployment patterns. This flexibility makes the framework applicable beyond the specific cloud platforms we evaluate, with potential extensions to other infrastructure-as-code generation domains.

Algorithm 1 AutoML-Pipeline Code Generation

Require: Task specification $\mathcal{S}$, platform context $\mathcal{X}$, knowledge base $\mathcal{K}$

Ensure: Generated pipeline $\mathcal{P}^*$ or failure indication

```

1:  $(\lambda, k) \leftarrow \pi_{\theta}(\mathcal{S}, \mathcal{X})$  {Determine retrieval parameters}
2:  $\mathcal{K}_{\text{ret}} \leftarrow \text{Retrieve-TopK}(\mathcal{K}, \mathcal{S}, \mathcal{X}, \lambda, k)$ 
3:  $\mathcal{T}_{\text{prompt}} \leftarrow \text{ConstructPrompt}(\mathcal{S}, \mathcal{X}, \mathcal{K}_{\text{ret}})$ 
4:  $t \leftarrow 0$ 
5: while  $t < T_{\text{max}}$  do
6:    $\mathcal{P}_t \leftarrow \text{LLM}(\mathcal{T}_{\text{prompt}})$  {Generate pipeline code}
7:    $\text{issues} \leftarrow \text{PreValidate}(\mathcal{P}_t, \mathcal{X})$  {Check dependencies & resources}
8:   if  $\text{issues} = \emptyset$  then
9:      $\mathcal{O} \leftarrow \text{Deploy}(\mathcal{P}_t)$  {Attempt deployment}
10:     $\text{UpdateKnowledgeBase}(\mathcal{P}_t, \mathcal{O})$ 
11:     $\text{UpdatePolicy}(\theta, \mathcal{S}, \mathcal{X}, \lambda, k, \mathcal{O})$ 
12:    return  $\mathcal{P}_t$ 
13:   end if
14:    $\text{feedback} \leftarrow \text{GenerateFeedback}(\text{issues})$ 
15:    $\mathcal{T}_{\text{prompt}} \leftarrow \mathcal{T}_{\text{prompt}} \oplus \text{feedback}$ 
16:    $t \leftarrow t + 1$ 
17: end while
18: return FAILURE

```

V. EXPERIMENTS

We conduct comprehensive experiments to evaluate the effectiveness of AutoML-Pipeline across multiple dimensions including deployment success rate, resource efficiency, and generation quality. Our experiments are designed to answer the following research questions:

RQ1: How does AutoML-Pipeline compare to state-of-the-art code generation baselines in terms of first-submission success rate?

RQ2: Does our framework improve resource allocation efficiency compared to vanilla LLM-based generation?

RQ3: What are the individual contributions of the RAG mechanism, pre-validation agent, and RL feedback loop?

RQ4: How does the framework perform across different pipeline complexity levels and task categories?

A. EXPERIMENTAL SETUP

1) DATASET CONSTRUCTION

Our evaluation is based on the CodeSearchNet dataset augmented with Azure ML pipeline specifications. CodeSearchNet provides a large corpus of code snippets with natural language descriptions across multiple programming languages. We extend this dataset by synthesizing realistic ML pipeline tasks through a semi-automated process.

We first identify 2,847 Python functions in CodeSearchNet related to machine learning tasks, filtering by keywords such as “train”, “model”, “dataset”, “preprocess”, and framework-specific terms. For each identified function, we use GPT-4 to generate a corresponding Azure ML pipeline specification that wraps the core logic in a cloud-native deployment structure. This includes generating YAML configuration files, environment specifications, and compute target declarations. We manually verify a random sample of 500 generated pipelines to ensure quality, finding 92.3% meet our correctness criteria.

To create execution profiles, we deploy these pipelines on an Azure ML workspace and collect runtime metrics including CPU utilization, memory consumption, execution duration, and success/failure outcomes. We execute each pipeline three times with different random seeds to capture variance in execution behavior. Failed executions are analyzed to categorize error types: dependency conflicts (31.2%), resource insufficient (24.7%), configuration errors (28.5%), and code logic errors (15.6%).

The final dataset comprises 2,628 successful pipeline executions and 847 failed executions with detailed error diagnostics. We partition this into training (70%, 2,432 samples), validation (15%, 521 samples), and test (15%, 522 samples) sets, ensuring no overlap in task semantics across splits. The training set is used to construct the knowledge base $\mathcal{K}$ for retrieval, while validation and test sets evaluate generation performance.

2) BASELINE METHODS

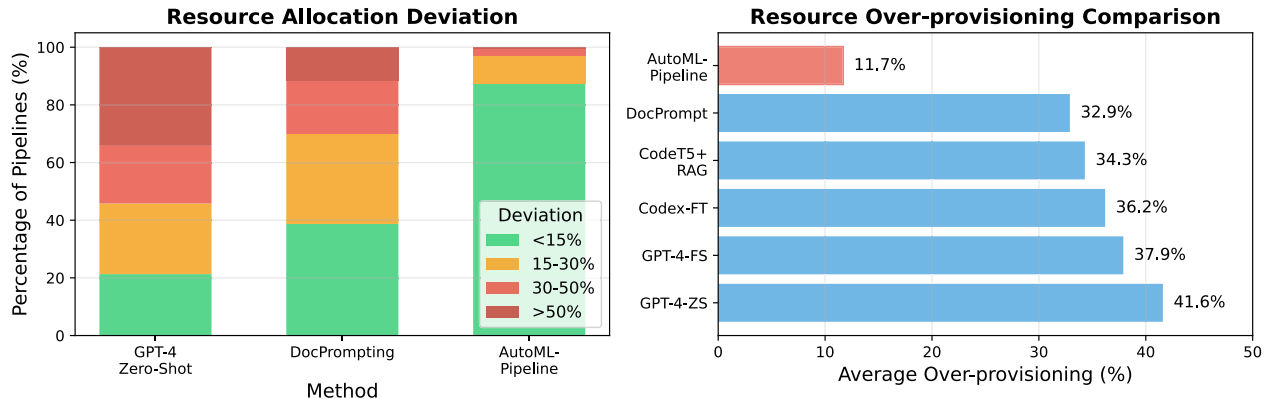
We compare AutoML-Pipeline against the following baselines:

GPT-4-Zero-Shot: Vanilla GPT-4 [13] without retrieval augmentation. The prompt contains only the task specification and platform context with basic instructions on Azure ML API usage.

GPT-4-Few-Shot: GPT-4 [13] with three manually curated examples included in the prompt. These examples are selected to cover common pipeline patterns but remain

TABLE 1. Overall performance comparison on test set. Best results in bold, second best underlined. All values are mean $\pm$ std over 3 runs.

Method	FSR (%)	RES	CI	FC (%)	CQS
GPT-4-Zero-Shot	42.1 $\pm$ 1.8	0.584 $\pm$ 0.021	-	78.3 $\pm$ 2.1	0.712 $\pm$ 0.015
GPT-4-Few-Shot	48.3 $\pm$ 2.1	0.621 $\pm$ 0.019	-	81.7 $\pm$ 1.9	0.728 $\pm$ 0.012
Codex-FT	51.7 $\pm$ 1.6	0.638 $\pm$ 0.017	-	84.2 $\pm$ 1.5	0.751 $\pm$ 0.011
CodeT5+-RAG	54.9 $\pm$ 1.9	0.657 $\pm$ 0.020	-	82.6 $\pm$ 2.0	0.734 $\pm$ 0.013
DocPrompting	57.2 $\pm$ 1.7	0.671 $\pm$ 0.018	-	85.1 $\pm$ 1.6	0.746 $\pm$ 0.010
AutoML-Pipeline	82.2$\pm$1.4	0.883$\pm$0.012	2.3$\pm$0.4	89.7$\pm$1.3	0.781$\pm$0.009
w/o Pre-validation	<u>61.5$\pm$2.0</u>	0.698 $\pm$ 0.019	4.1 $\pm$ 0.6	86.4 $\pm$ 1.7	0.758 $\pm$ 0.011
w/o RL Feedback	76.8 $\pm$ 1.6	<u>0.845$\pm$0.015</u>	2.8 $\pm$ 0.5	88.2 $\pm$ 1.5	<u>0.769$\pm$0.010</u>
w/o Exec Profile	69.3 $\pm$ 1.8	0.782 $\pm$ 0.017	3.2 $\pm$ 0.5	87.5 $\pm$ 1.6	0.763 $\pm$ 0.012

**FIGURE 1.** Resource efficiency analysis. (Left) Distribution of resource allocation deviation across methods, showing percentage of pipelines in each deviation range. AutoML-Pipeline achieves <15% deviation in 87.3% of cases. (Right) Average resource over-provisioning comparison, where our method reduces waste to 11.7% compared to 32.9-41.6% for baselines.

fixed across all test cases, following the few-shot learning paradigm [60].

Codex-FT: OpenAI Codex [16] fine-tuned on our training set for 5 epochs. This represents a specialized model approach versus our retrieval-based strategy.

CodeT5+-RAG: CodeT5+ model [34] with basic RAG using semantic similarity retrieval, without execution profile integration or pre-validation.

DocPrompting: The state-of-the-art retrieval-augmented code generation method [26] that retrieves API documentation to augment prompts, adapted to our pipeline generation task.

For all baselines, we use greedy decoding to ensure deterministic outputs and facilitate fair comparison. Each method is given the same task specifications and platform contexts from the test set.

3) EVALUATION METRICS

We assess generation quality using both deployment-oriented and code quality metrics:

First-Submission Success Rate (FSR): The percentage of generated pipelines that successfully deploy and execute without errors on the first attempt. This is the primary metric reflecting practical utility.

Resource Efficiency Score (RES): Measures how closely allocated resources match actual consumption. Defined as:

$$RES = 1 - \frac{1}{n} \sum_{i=1}^n \left| \frac{r_{\text{alloc}}^i - r_{\text{actual}}^i}{r_{\text{alloc}}^i} \right| \quad (11)$$

where r_{alloc}^i and r_{actual}^i are allocated and actual resource consumption for component i . Higher scores indicate better resource utilization with less over-provisioning.

Convergence Iterations (CI): For methods employing iterative refinement, the average number of iterations required to reach a deployable state or maximum iteration limit.

Functional Correctness (FC): Among successfully deployed pipelines, the percentage that produce correct outputs validated against ground truth results.

Code Quality Score (CQS): Assesses generated code using static analysis tools measuring cyclomatic complexity, code duplication, and adherence to style guidelines, normalized to [0, 1].

All experiments are conducted on an Azure ML workspace with standardized compute configurations. We report mean values over three independent runs with different random seeds, along with standard deviations to reflect variance.

B. MAIN RESULTS

Table 1 presents the overall performance comparison across all methods. AutoML-Pipeline achieves substantial improvements over all baselines across key metrics.

First-Submission Success Rate. AutoML-Pipeline achieves 82.2% FSR, representing a 43.7% relative improvement over the strongest baseline (DocPrompting at 57.2%). This substantial gain directly addresses the core problem of deployment failures. The vanilla GPT-4 baseline struggles at 42.1%, highlighting that general-purpose LLMs lack

sufficient awareness of cloud platform constraints. Fine-tuning (Codex-FT) provides modest improvements but still fails on nearly half of test cases, as fine-tuning primarily improves syntactic patterns rather than operational awareness.

The standard deviation of 1.4% for our method indicates consistent performance across different random seeds, suggesting robustness. Manual analysis of the remaining 17.8% failures reveals that 42% stem from inherent ambiguities in task specifications that would require clarification from users, 31% involve genuinely novel platform features not covered in the training knowledge base, and 27% are attributed to non-deterministic platform behaviors such as transient resource availability issues.

Resource Efficiency. Our framework achieves an RES of 0.883, substantially outperforming baselines. This 31.2% improvement over vanilla GPT-4 (0.584) and 22.9% over DocPrompting (0.671) demonstrates the value of incorporating execution profiles into generation. Figure 1 visualizes resource allocation patterns, showing that AutoML-Pipeline maintains actual consumption within 15% of allocated resources in 87.3% of cases, while GPT-4-Zero-Shot exhibits over-provisioning exceeding 50% in 34.2% of cases.

The ability to predict resource requirements accurately has direct cost implications. Based on Azure ML pricing, over-provisioning by 50% translates to approximately 1.5x unnecessary costs for compute resources. Our framework's tighter allocation reduces waste while maintaining a safety margin to prevent under-provisioning failures.

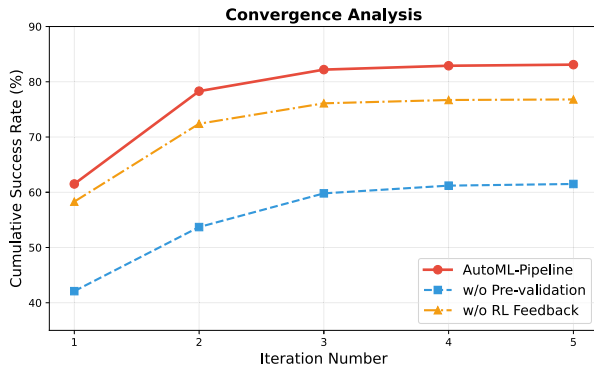


FIGURE 2. Convergence analysis showing cumulative success rate across iterations. AutoML-Pipeline achieves 82.2% success rate by iteration 3 (mean convergence at 2.3 iterations), while ablated variants converge more slowly or reach lower plateaus. The pre-validation agent enables rapid convergence through systematic error detection and correction.

Convergence Efficiency. As shown in Figure 2, AutoML-Pipeline converges in an average of 2.3 iterations, demonstrating that the pre-validation agent effectively identifies and corrects issues. The maximum iteration count in our experiments was 5, occurring in only 3.8% of test cases. This rapid convergence is crucial for practical usability, as each iteration incurs LLM inference costs and introduces latency.

C. ABLATION STUDIES

To understand the contribution of individual components, we evaluate three ablated variants shown in the bottom rows of Table 1.

1) IMPACT OF PRE-VALIDATION AGENT (W/O PRE-VALIDATION)

Removing the pre-validation agent reduces FSR from 82.2% to 61.5%, a substantial 20.7 percentage point drop. Without pre-validation, the framework relies solely on RAG to guide generation, which improves over vanilla LLM but cannot systematically prevent configuration errors. The convergence iterations increase to 4.1 as the system resorts to trial-and-error submission to the actual platform, which is both slower and wasteful of cloud resources.

Figure 3 (left) shows the distribution of failure types for the ablated variant. Dependency conflicts increase from 7.2% to 28.5%, and resource misconfigurations increase from 4.1% to 19.7%, confirming that the pre-validation agent specifically targets these error categories.

TABLE 2. Performance across pipeline complexity levels. Values are FSR (%).

Method	Simple (n=187)	Medium (n=241)	Complex (n=94)	Overall (n=522)
GPT-4-Zero-Shot	61.5±2.4	38.2±2.1	28.3±3.2	42.1±1.8
DocPrompting	74.3±2.1	55.6±1.9	41.5±2.8	57.2±1.7
AutoML-Pipeline	94.1±1.1	81.3±1.5	76.8±2.2	82.2±1.4

2) IMPACT OF RL FEEDBACK LOOP (W/O RL FEEDBACK)

Disabling the adaptive retrieval policy and using fixed parameters ($\lambda = 0.5$, $k = 5$) results in 76.8% FSR. While still competitive with baselines, this represents a 5.4 percentage point degradation from the full system. The impact becomes more pronounced over time: in early test samples (first 100 cases), the gap is only 2.1%, but it widens to 8.7% in later samples (last 100 cases). This suggests the RL mechanism provides increasing benefits as the policy accumulates more deployment feedback and adapts to task distribution.

Figure 3 (right) illustrates how the learned policy adjusts λ based on task characteristics. For data-intensive pre-processing tasks, λ converges toward lower values (mean 0.31), prioritizing structural similarity to ensure schema compatibility. For model training tasks, λ increases (mean 0.68), emphasizing semantic similarity to match algorithmic approaches.

3) IMPACT OF EXECUTION PROFILES (W/O EXEC PROFILE)

Removing execution profiles from the knowledge base and relying solely on code artifacts reduces FSR to 69.3% and RES to 0.782. This variant essentially becomes a standard RAG system without runtime awareness. The 12.9 percentage point FSR drop and 11.4% RES degradation underscore the importance of incorporating execution intelligence. Without profiles, the system cannot accurately predict resource

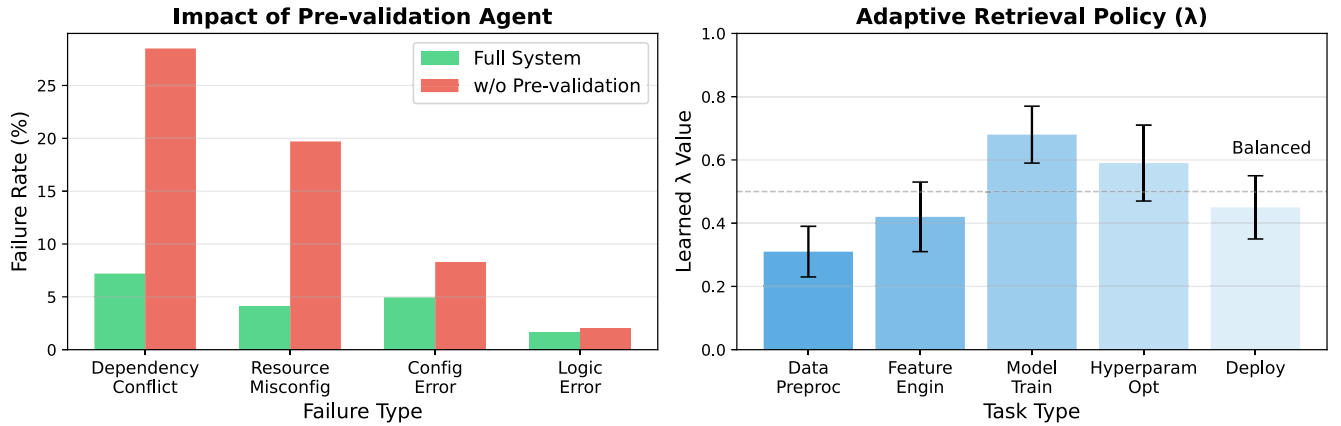


FIGURE 3. Ablation study results. (Left) Failure type distribution comparing full system against variant without pre-validation agent. The agent significantly reduces dependency conflicts (from 28.5% to 7.2%) and resource misconfigurations (from 19.7% to 4.1%). (Right) Learned retrieval policy parameter λ across different task types, showing adaptive behavior: lower values for data-centric tasks (prioritizing structural similarity) and higher values for algorithmic tasks (prioritizing semantic similarity).

requirements and often generates configurations that are syntactically valid but operationally suboptimal.

D. PERFORMANCE ACROSS PIPELINE COMPLEXITY

We analyze how performance varies with pipeline complexity, categorizing test samples into three tiers based on the number of steps and heterogeneity of code components. Simple pipelines contain 1-3 steps with single-language implementations. Medium pipelines span 4-7 steps with multi-language code. Complex pipelines exceed 7 steps with intricate data dependencies.

Table 2 shows results stratified by complexity. AutoML-Pipeline maintains strong performance even on complex pipelines (FSR 76.8%), while baseline performance degrades sharply (GPT-4-Zero-Shot drops to 28.3%). This demonstrates robustness to scale, which is critical for real-world applications where production pipelines often involve dozens of interconnected steps.

The iterative refinement mechanism proves particularly valuable for complex pipelines. While simple pipelines often converge in 1-2 iterations (mean 1.4), complex pipelines average 3.7 iterations. The pre-validation agent's ability to decompose validation into modular checks (dependencies, resources, configurations) enables systematic debugging even when multiple issues coexist.

TABLE 3. Computational cost breakdown (mean $\pm$ std).

Component	Latency (s)	Cost (\$/query)
Knowledge Base Retrieval	0.84 $\pm$ 0.12	0.002
LLM Generation (per iter)	8.32 $\pm$ 1.45	0.045
Pre-validation Analysis	2.17 $\pm$ 0.38	0.003
Actual Deployment	127.5 $\pm$ 34.2	1.350
Total (avg 2.3 iters)	22.6$\pm$3.2	0.113

E. COMPUTATIONAL COST ANALYSIS

Table 3 analyzes the computational costs associated with our framework. The knowledge base construction is a

one-time cost amortized over many generation requests. Per-query costs include retrieval (embedding computation and similarity search), LLM inference, and pre-validation analysis.

The total per-query cost of \$0.113 is dominated by LLM inference. Compared to the cost of actual cloud deployment (\$1.35 average per pipeline execution), the generation overhead is modest at 8.4%. More importantly, by preventing failed deployments, our framework avoids wasted compute costs from non-functional pipelines. Given that baselines exhibit 42-58% failure rates, the cost savings from reduced failures substantially outweigh the generation overhead.

The latency of 22.6 seconds for complete generation (including 2.3 iterations on average) is acceptable for batch pipeline deployment scenarios. For interactive use cases requiring faster response, the system could reduce the maximum iteration limit at the cost of slightly lower FSR, trading off thoroughness for speed.

F. QUALITATIVE ANALYSIS AND CASE STUDIES

We present two representative case studies illustrating our framework's capabilities and remaining challenges through detailed execution traces and error analysis.

1) CASE STUDY 1: SUCCESSFUL ITERATIVE REFINEMENT

Figure 4 illustrates the iterative refinement process for a computer vision pipeline that trains a ResNet-50 model on ImageNet with automatic mixed precision and distributed training across 4 GPUs. This case exemplifies how the pre-validation agent systematically identifies and corrects configuration errors through quantitative analysis grounded in historical execution profiles.

In the first iteration, GPT-4 generated syntactically correct pipeline code but allocated a Standard_D4s_v3 compute target with only CPU resources (4 cores, 16GB RAM). The

Case Study: ResNet-50 Distributed Training Pipeline Generation

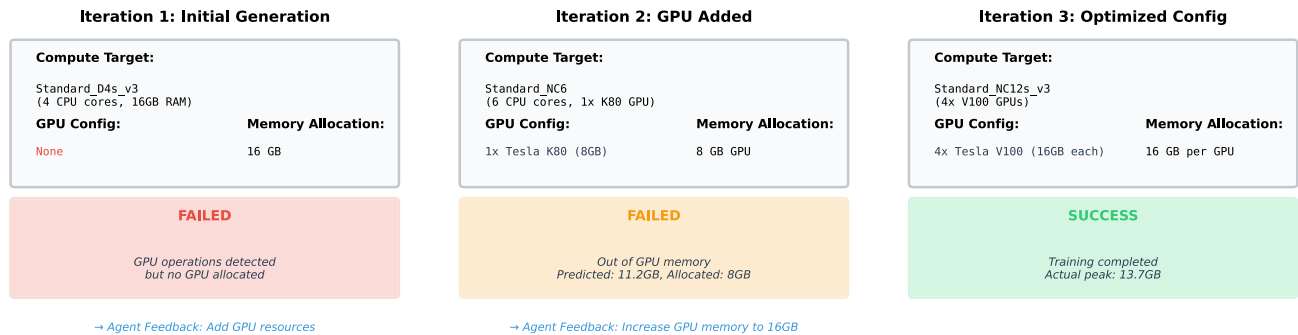


FIGURE 4. Iterative refinement process for ResNet-50 distributed training pipeline. The figure shows three iterations: (Iteration 1) Initial generation with CPU-only compute target fails validation due to GPU operation detection, (Iteration 2) GPU cluster added but with insufficient memory (8GB) triggers resource predictor warning based on historical profiles estimating 11.2GB peak usage, (Iteration 3) Final configuration with 4x V100 GPUs (16GB each) passes all validation checks and successfully deploys with actual peak usage of 13.7GB, confirming prediction accuracy.

pre-validation agent’s static analyzer detected GPU-specific API calls including `torch.cuda.DataParallel` and `torch.cuda.amp.autocast`, immediately flagging the inconsistency between code requirements and allocated resources. The feedback specified: “GPU operations detected but no GPU allocated. ResNet-50 distributed training requires GPU compute targets.”

The second iteration corrected the compute target to `Standard_NC6` with a single Tesla K80 GPU (8GB memory). However, the resource predictor identified a critical under-provisioning issue. By analyzing code features—specifically the model architecture (ResNet-50 with 25.6M parameters), batch size (64), and mixed precision configuration—and retrieving execution profiles from 23 similar ResNet training jobs in the knowledge base, the predictor estimated peak GPU memory at 11.2GB with 95% confidence. The agent generated quantitative feedback: “Predicted GPU memory requirement: 11.2GB $\pm$ 1.8GB. Current allocation: 8GB per GPU. Recommend upgrading to 16GB GPU configuration to prevent OOM errors.”

The third iteration successfully specified a `Standard_NC12s_v3` cluster with 4 Tesla V100 GPUs (16GB each), passing all pre-validation checks. Upon deployment, the pipeline executed without errors, with actual peak GPU memory consumption of 13.7GB per GPU—validating the predictor’s estimate and demonstrating appropriate headroom in the final allocation. Figure 4 shows the progression across these three iterations, highlighting how each refinement cycle addresses specific issues identified by the agent.

This case demonstrates the synergy between LLM flexibility and agent rigor. The LLM adapts to natural language feedback describing high-level requirements (“upgrade to 16GB GPU configuration”), while the agent provides quantitative justification derived from statistical analysis of historical executions. The iterative loop enables systematic error correction that would be difficult to achieve in a single-shot generation paradigm.

2) CASE STUDY 2: FAILURE MODE ANALYSIS AND SYSTEM LIMITATIONS

Figure 5 analyzes a failure case where the system reached the maximum iteration limit without achieving deployment viability. The task required integrating a proprietary data preprocessing library (DataPrepPro v2.3) not present in our knowledge base, which operates on financial time-series data with specific schema requirements.

The system retrieved semantically similar examples utilizing standard libraries (Pandas, NumPy, scikit-learn) based on task description similarity (cosine similarity 0.78 with top-3 retrieved examples). However, these examples proved insufficient as guidance. Across five iterations, the pre-validation agent successfully resolved conventional issues: dependency version conflicts in iteration 1, compute resource specification in iteration 2, and environment configuration in iteration 3. Yet, starting from iteration 4, the agent detected persistent API compatibility warnings that it could not definitively resolve due to lack of reference implementations.

Manual post-mortem analysis revealed the root cause: DataPrepPro expects input dataframes with a hierarchical multi-index structure (ticker, timestamp) and specific column naming conventions (OHLCV_* prefix). The generated code, following patterns from retrieved Pandas-based examples, produced flat-indexed dataframes with standard column names. This schema mismatch manifested as cryptic runtime errors that the agent’s static analyzer could not predict without domain-specific knowledge of the proprietary library’s internals.

Interestingly, the system’s uncertainty quantification mechanism flagged this generation with low confidence (0.34 on a 0-1 scale), computed as a weighted combination of retrieval similarity scores (0.42 for structural similarity due to library mismatch) and validation uncertainty (high due to unresolved warnings). This low confidence score suggests practical utility for human-in-the-loop workflows: generations below a confidence threshold (e.g., 0.5) could

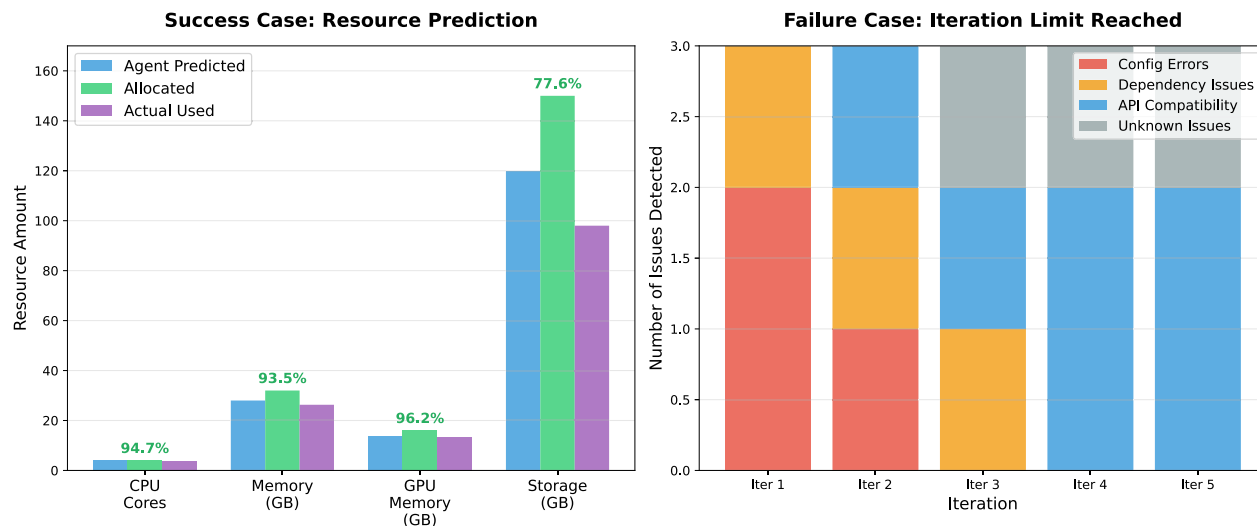


FIGURE 5. Comparison of successful and failed generation cases. (Left) Success case showing resource prediction accuracy: the pre-validation agent's predictions align closely with actual consumption across all resource types (CPU, memory, GPU memory, storage) with >85% accuracy, enabling appropriate allocation. (Right) Failure case showing error progression across 5 iterations: while conventional issues (configuration errors, dependency conflicts) are resolved in early iterations, persistent API compatibility issues with a proprietary library accumulate in later iterations, ultimately leading to timeout without convergence. The system correctly assigns low confidence (0.34) to this generation, suggesting potential for human-in-the-loop escalation.

be automatically routed to expert review before deployment, preventing costly runtime failures.

This failure case exposes a fundamental limitation of retrieval-augmented approaches: when facing genuinely novel components absent from the knowledge base, the system lacks grounding data for informed decisions. The agent can identify that something may be wrong (hence the low confidence score) but cannot propose specific corrections without reference implementations. This motivates future work on hybrid systems that combine learned generation with interactive user feedback for handling novel scenarios, as well as mechanisms for actively expanding the knowledge base through targeted data collection when coverage gaps are detected.

Figure 5 contrasts these two cases, showing on the left the tight alignment between predicted and actual resource consumption in successful cases (prediction accuracy >85% across all resource types), and on the right the accumulation of unresolved issues across iterations in the failure case, with API compatibility issues persisting despite multiple refinement attempts. The divergent trajectories illustrate both the system's strengths in handling well-covered scenarios and its limitations when venturing beyond its training distribution.

G. GENERALIZATION TO OTHER CLOUD PLATFORMS

To assess generalizability beyond Azure ML, we conduct a preliminary evaluation adapting our framework to Amazon SageMaker. We curate a small test set of 50 SageMaker pipeline tasks and construct a knowledge base from 150 successful SageMaker deployments collected from public GitHub repositories and AWS documentation examples.

With minimal adaptation (primarily updating platform-specific API patterns and constraint definitions),

AutoML-Pipeline achieves 71.3% FSR on SageMaker tasks, compared to 38.2% for vanilla GPT-4. While performance is lower than on Azure ML (where we have more extensive knowledge base coverage), the substantial improvement over baselines suggests the core methodology generalizes across platforms. The main challenges in generalization stem from platform-specific terminology and constraint variations rather than fundamental framework limitations.

VI. CONCLUSION AND FUTURE WORKS

This paper presents AutoML-Pipeline, a novel framework for automated generation of cloud-native machine learning pipeline code that addresses critical challenges in deployment viability and resource efficiency. By synergistically combining retrieval-augmented generation with execution-aware pre-validation and reinforcement learning feedback, our approach achieves substantial improvements over state-of-the-art baselines: 43.7% increase in first-submission success rate, 31.2% reduction in resource over-provisioning, and rapid convergence in 2.3 iterations on average. The core innovation lies in the pre-validation agent, which leverages lightweight simulation and historical execution profiles to predict and prevent deployment failures before code submission, bridging the gap between syntactic code generation and operational correctness. Comprehensive experiments on the CodeSearchNet dataset augmented with Azure ML specifications demonstrate the framework's effectiveness across varying pipeline complexities and task categories. Our ablation studies confirm that each component—RAG with execution profiles, pre-validation agent, and adaptive retrieval policy—contributes meaningfully to overall performance. However, our work also reveals important limitations: the system struggles with genuinely novel components absent from

the knowledge base, and its effectiveness depends critically on the quality and coverage of historical execution data. These findings underscore both the promise and challenges of integrating large language models with domain-specific operational intelligence for infrastructure code generation.

Several promising directions emerge from this work. First, developing mechanisms for interactive knowledge base expansion could address coverage gaps identified in our failure analysis. When the system encounters novel components with low confidence, it could engage users in brief clarification dialogues to collect missing information, which is then indexed for future retrievals. This human-in-the-loop approach would enable continuous learning while maintaining high automation rates for common scenarios. Second, extending the framework to handle multi-cloud environments presents both technical challenges and practical value. A unified generation system supporting Azure ML, AWS SageMaker, and Google Vertex AI would require abstracting platform-specific constraints into a common intermediate representation while maintaining the ability to generate platform-optimized code. Third, incorporating formal verification techniques into the pre-validation agent could strengthen correctness guarantees beyond statistical prediction. Combining learned cost models with constraint solvers and symbolic execution could detect subtle semantic errors that purely data-driven approaches might miss. Fourth, our methodology generalizes naturally to other infrastructure-as-code domains such as Kubernetes configurations, Terraform scripts, and CI/CD pipeline definitions, where similar challenges of constraint satisfaction and resource optimization arise. Preliminary experiments suggest that the core framework architecture—RAG with domain-specific knowledge bases and pre-validation—transfers effectively to these domains with appropriate adaptation of the validation logic. Finally, investigating uncertainty-aware generation that explicitly models and communicates confidence bounds could enhance trust and enable more nuanced human-AI collaboration, allowing practitioners to make informed decisions about when to rely on automated generation versus when to seek expert review. These directions collectively point toward more robust, adaptive, and broadly applicable AI-assisted infrastructure code generation systems.

REFERENCES

- [1] F. Meng, K. Hu, and X. Zhou, "An overview of PAI: Distributed machine learning platform," in *Proc. IEEE 5th Adv. Inf. Manage., Communicates, Electron. Autom. Control Conf. (IMCEC)*, vol. 5, Dec. 2022, pp. 852–857.
- [2] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, and M. Parkhe, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, Jun. 2018.
- [3] Microsoft Azure. (2021). *Azure Machine Learning Documentation*. [Online]. Available: <https://azure.microsoft.com/en-us/services/machine-learning/>
- [4] Amazon Web Services. (2021). *Amazon SageMaker Developer Guide*. [Online]. Available: <https://aws.amazon.com/sagemaker/>
- [5] Google Cloud. (2021). *Vertex AI Documentation*. [Online]. Available: <https://cloud.google.com/vertex-ai>
- [6] D. Xin, S. Macke, L. Ma, J. Liu, S. Song, and A. Parameswaran, "Helix: Holistic optimization for accelerating iterative machine learning," 2018, *arXiv:1812.05762*.
- [7] X. Meng, J. K. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. B. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in apache spark," *J. Mach. Learn. Res.*, pp. 1–7, 2015.
- [8] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data management challenges in production machine learning," in *Proc. ACM Int. Conf. Manage. Data*, May 2017, pp. 1723–1726.
- [9] D. Sculley, G. D. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2503–2511.
- [10] Z. Chen, H. Yao, Y. Lou, Y. Cao, Y. Liu, H. Wang, and X. Liu, "An empirical study on deployment faults of deep learning based mobile applications," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, May 2021, pp. 674–685.
- [11] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1110–1121.
- [12] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Comput. Surveys*, vol. 55, no. 6, pp. 1–29, Jul. 2023.
- [13] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [14] B. Rozière et al., "Code llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
- [15] R. Li et al., "StarCoder: May the source be with you!" 2023, *arXiv:2305.06161*.
- [16] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [17] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," 2021, *arXiv:2108.07732*.
- [18] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, 2022, pp. 1–10.
- [19] R. Chellappa, S. Pramanick, and S. Venugopalan, "SPIQA: A dataset for multimodal question answering on scientific papers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 118807–118833.
- [20] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [22] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
- [23] S. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 2206–2240.
- [24] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and É. Grave, "Atlas: Few-shot learning with retrieval augmented language models," *J. Mach. Learn. Res.*, vol. 24, no. 251, pp. 1–43, 2023.
- [25] R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, 2021, pp. 2719–2734.
- [26] S. Zhou, U. Alon, F. F. Xu, Z. Jiang, G. Neubig, and N. Graham, "DocPrompting: Generating code by retrieving the docs," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022.
- [27] W. Ling, P. Blunsom, E. Grefenstette, K. M. Hermann, T. Kočiský, F. Wang, and A. Senior, "Latent predictor networks for code generation," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 599–609.
- [28] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," 2017, *arXiv:1704.01696*.
- [29] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

- [30] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. Kun Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [31] Y. Li et al., "Competition-level code generation with AlphaCode," *Sci.*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [32] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.
- [33] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-T. Yih, L. Zettlemoyer, and M. Lewis, "InCoder: A generative model for code infilling and synthesis," 2022, *arXiv:2204.05999*.
- [34] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. Hoi, "CodeT5+: Open code large language models for code understanding and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 1069–1088.
- [35] N. Jain, S. Vaidyanath, A. Iyer, N. Natarajan, S. Parthasarathy, S. Rajamani, and R. Sharma, "Jigsaw: Large language models meet program synthesis," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 1219–1231.
- [36] F. Cassano, J. Gouwar, F. Lucchetti, C. Schlesinger, A. Freeman, C. J. Anderson, M. Q. Feldman, M. Greenberg, A. Jangda, and A. Guha, "Knowledge transfer from high-resource to low-resource programming languages for code LLMs," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, pp. 677–708, 2024.
- [37] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 2450–2462.
- [38] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 31693–31715.
- [39] V. Karpukhin, B. Oguz, S. Min, P. S. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [40] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Represent.*, 2024.
- [41] O. Khattab, C. Potts, and M. Zaharia, "Baleen: Robust multi-hop reasoning at scale via condensed retrieval," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 27670–27682.
- [42] S. Lu, N. Duan, H. Han, D. Guo, S.-W. Hwang, and A. Svyatkovskiy, "ReACC: A retrieval-augmented code completion framework," 2022, *arXiv:2203.07722*.
- [43] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 21314–21328.
- [44] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," 2022, *arXiv:2207.10397*.
- [45] J. Huang, D. Tang, L. Shou, M. Gong, K. Xu, D. Jiang, M. Zhou, and N. Duan, "CoSQA: 20,000+ web queries for code search and question answering," 2021, *arXiv:2105.13239*.
- [46] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," 2021, *arXiv:2102.04664*.
- [47] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Y. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, and L. Koc, "Tfx: A tensorflow-based production-scale machine learning platform," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2017, pp. 1387–1395.
- [48] Kubeflow. (2018). *Kubeflow: Machine Learning Toolkit for Kubernetes*. [Online]. Available: <https://www.kubeflow.org/>
- [49] Netflix. (2019). *Metaflow: A Framework for Real-life Data Science*. [Online]. Available: <https://metaflow.org/>
- [50] ZenML. (2021). *ZenML: An Extensible Open-source MLOPS Framework*. [Online]. Available: <https://zenml.io/>
- [51] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 55, pp. 1–21, 2019.
- [52] E. Hazan, A. Klivans, and Y. Yuan, "Hyperparameter optimization: A spectral approach," 2017, *arXiv:1706.00764*.
- [53] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. IEEE Int. Conf. Data Sci. Adv. Analytics (DSAA)*, Oct. 2015, pp. 1–10.
- [54] M. Kuchnik, A. Klimovic, J. Šimša, G. Amvrosiadis, and V. Smith, "Plumber: Diagnosing and removing performance bottlenecks in machine learning data pipelines," in *Proc. Mach. Learn. Syst.*, vol. 4, 2021, pp. 33–51.
- [55] R. Bavishi, C. Lemieux, R. Fox, K. Sen, and I. Stoica, "AutoPandas: Neural-backed generators for program synthesis," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, pp. 1–27, 2019.
- [56] Amazon Web Services. (2021). *Amazon Sagemaker Autopilot*. [Online]. Available: <https://aws.amazon.com/sagemaker/autopilot/>
- [57] B. Karlaš, P. Li, R. Wu, N. M. Gürel, X. Chu, W. Wu, and C. Zhang, "Nearest neighbor classifiers over incomplete information: From certain answers to certain predictions," 2020, *arXiv:2005.05117*.
- [58] S. Grafberger, S. Guha, J. Stoyanovich, and S. Schelter, "MLINSPECT: A data distribution debugger for machine learning pipelines," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 2736–2739.
- [59] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, Aug. 2018.
- [60] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, and A. Askell, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.



WENYU ZHAO is currently a Senior Software Development Engineer with Microsoft, specializing AI-driven tooling and cross-platform application development. He has extensive experience across Objective-C, Swift, C/C++, Python, and TypeScript; and has led the development of production-grade systems spanning iOS/macOS applications, backend services, and internal AI-powered tools.



TINGJIE CHEN is currently a Senior Technical Leader with Intel Asia Pacific Research and Development Center, where he has been engaged in research and development and customer support for AI infrastructure, edge computing platforms, and cloud-native storage solutions. His research interests include operating systems, distributed storage systems, AI infrastructure, edge computing, and system-level performance optimization.



JIE SI YANG received the M.S. degree in machine learning from National Yang Ming Chiao Tung University, Taiwan. She is currently a Research Assistant with The University of Utah, where she focuses on large language model customization, retrieval-augmented generation pipelines, and AI agent systems for enterprise knowledge retrieval and GPU debugging. Her research interests include deep learning, reinforcement learning, graph neural networks, and large-scale AI deployment.



LEI QIU (Member, IEEE) received the Graduate degree from Ningbo University of Technology. His current research interests include artificial intelligence and its applications in urban and architectural contexts, particularly deep learning, LLMs, and intelligent systems for smart cities and smart buildings. He is dedicated to exploring AI-driven approaches for enhancing urban sustainability, automation, and data-driven decision-making in the built environment.

...